Stop When Stable: Adaptive GPU Profiling for LLM Serving

***Abstract*—GPU profilers expose the kernel-level timing needed to diagnose LLM-serving slowdowns, but they are usually run manually for fixed windows. These windows are often chosen by convention: short runs can miss a stable bottleneck label, while long runs collect redundant traces after the diagnosis has converged.**

We present an *adaptive GPU tracing controller* that turns profiler use into a runtime decision. The controller keeps low-cost request and GPU counters on continuously, starts short Nsight Systems bursts when cheap signals are insufficient, and stops heavy collection when the bottleneck label is stable. We evaluate six stopping policies on NVIDIA L4 and A100 GPUs using vLLM with `Qwen2.5-7B-Instruct`, six serving scenarios, and three controlled microbenchmarks. Relative to a fixed-window baseline, adaptive tracing reduces profiler duration by 18.9%–48.5% and captured kernel instances by 42.9%–78.2% while preserving expected-label agreement in the controlled workloads. Cheap always-on metrics stay within the 5% p95-latency overhead threshold in dedicated perturbation runs. Under an ambiguity stress replay, one fixed burst stops too early, while recovery-gated policies avoid re-escalation at higher tracing cost. Among the tested signals, two-window diagnosis stability is the safest standalone stopping criterion.

I. Introduction

Large language model (LLM) serving systems must sustain high throughput and predictable latency in production, yet they are routinely disrupted by performance anomalies: request queueing [1], compute saturation [2], memory pressure, long input or output sequences, or transient workload shifts. Diagnosing the cause often requires kernel-level evidence: execution time, launch frequency, and duration distributions from a GPU profiler such as NVIDIA Nsight Systems. Cheap runtime counters can show that performance changed, but they rarely identify the kernel-level pattern behind the change. This kernel-level evidence is expensive to collect, so operators typically invoke the profiler manually with a fixed time window chosen by convention rather than by the state of the diagnosis.

This creates a tension between profiling cost and diagnostic confidence. A fixed-window profiler that runs too briefly may stop before the bottleneck class has stabilized into a concrete diagnosis, while one that runs too long wastes profiler time and storage recording redundant kernel evidence after the diagnosis has already converged. Existing practice offers no principled way to decide, while a profiling session is active, whether enough evidence has been collected to stop.

We address this gap with an *adaptive GPU tracing controller* that automatically manages heavy GPU profiling for LLM serving systems, building on runtime-adaptive evidence escalation [3], [4]. This work treats GPU profiling as an automated debugging-support problem: when a serving system enters an anomalous state, the controller decides what evidence a developer would otherwise collect manually, when to collect it, and when enough evidence has been gathered. Cheap signals are collected continuously, while short Nsight Systems bursts are activated only when cheap evidence is insufficient and stopped once the kernel-level diagnosis stabilizes. Here, *adaptive* describes a deterministic finite-state machine governed by fixed, hand-tuned thresholds; no model is trained or fitted online. We design and evaluate six short-burst *stopping policies* and identify which runtime signals most reliably indicate evidence sufficiency.

We evaluate the controller on NVIDIA L4 and A100 GPUs using vLLM serving `Qwen2.5-7B-Instruct` across six serving scenarios and three controlled microbenchmark workloads, guided by five research questions:

- **RQ1:** Can the controller determine when sufficient diagnostic evidence has been collected, stopping heavy tracing earlier than a fixed-window baseline without reducing expected-label agreement?
- **RQ2:** How accurate is automatic GPU tracing compared with manual fixed-window profiling?
- **RQ3:** What runtime perturbation is introduced by the always-on cheap metrics, and how much profiler cost does adaptive tracing avoid?
- **RQ4:** Which short-burst GPU tracing stopping policies work best, and what tradeoffs do they make between cost and reliability?
- **RQ5:** Which runtime signals are most useful for deciding when to stop heavy GPU tracing?

A. Contributions

This paper makes the following contributions:

1) A debugging-support formulation of GPU trace stopping: the controller decides when profiler evidence is needed and when additional bursts are unlikely to change the bottleneck label.
2) A rule-based adaptive tracing controller implemented as a finite-state machine over cheap request metrics, NVML counters, and short Nsight Systems bursts.
3) A comparison of six stopping policies using premature-stop rate, re-escalation rate, trace duration, and expected-label agreement.
4) An evaluation on L4 and A100 GPUs showing 18.9%–48.5% lower profiler duration and 42.9%–78.2% fewer captured kernel instances than a fixed-window baseline in the controlled workloads.

5) A signal analysis showing that two-window diagnosis stability is the only tested standalone signal that avoids ambiguous early stops.

The remainder of the paper reviews related work, describes the controller and stopping policies, presents the evaluation, discusses scope and threats to validity, and concludes.

II. Related Work

Work relevant to adaptive, budget-aware GPU tracing spans three bodies of literature: distributed tracing systems that decide what request-level evidence to collect, GPU profiling tools that expose accelerator-level evidence, and GPU-heavy LLM serving systems that produce the ambiguous bottlenecks our controller diagnoses.

a) Adaptive tracing.: Production tracing systems such as Dapper [5], X-Trace [6], and Canopy [7] established always-on, low-cost request-level tracing, but treat neither GPU execution nor kernel launches as first-class evidence. Recent tracing systems make collection decisions at runtime. TraStrainer [8] uses trace diversity and runtime state to guide distributed-trace sampling, and Astraea [9] treats span selection as a budgeted online decision problem. These systems adapt request-trace collection, while our controller adapts a separate GPU-profiler tier: it starts, extends, and stops kernel-level bursts based on evidence sufficiency. Neither, however, treats GPU profiler activation or kernel-level tracing as a tiered evidence source with its own escalation and de-escalation policy.

Both adapt a single scalar per-request sampling rate over distributed trace spans rather than activating a discrete GPU evidence tier, so we treat them as conceptually related rather than as ported baselines and leave a learned, budget-constrained GPU stopping policy as future work.

b) GPU profiling tools.: NVBit [10] provides dynamic binary instrumentation for fine-grained GPU kernel tracing, representative of the heaviest tier in our evidence hierarchy. HPCToolkit [11] and top-down GPU profilers [12], [13] show how GPU tools can expose counters, samples, and kernel-level performance structure. These tools help interpret GPU behavior after evidence has been collected; they do not decide when an LLM-serving incident has enough kernel evidence to stop profiling. Both, however, assume a human operator or an offline workflow decides when to invoke the profiler and for how long; neither includes a runtime policy that activates, extends, or stops collection based on diagnostic sufficiency.

c) LLM serving systems.: vLLM's PagedAttention [14] demonstrates how GPU memory pressure, batching, and request latency interact during LLM serving, producing exactly the ambiguous, multi-cause bottlenecks that justify escalating to kernel-level evidence and that make premature de-escalation risky. We use vLLM as the concrete serving substrate on which our controller is evaluated.

Our work sits at the intersection of these three areas: it adopts the adaptive, budget-aware control philosophy of systems such as TraStrainer and Astraea, applies it to the GPU evidence sources exposed by tools such as NVBit and HPCToolkit, and evaluates it on the realistic, ambiguous bottlenecks produced by LLM serving systems such as vLLM. We are not aware of prior work that studies runtime policies for starting, extending, and stopping costly GPU tracing while preserving diagnosis quality.

III. Methodology

A. System Overview

The controller runs once per evidence window and manages one anomaly episode at a time. An episode begins when cheap T0/T1 signals mark a window as suspicious and ends when the active stopping policy de-escalates to cheap-only monitoring. During an episode, the controller may launch one or more bounded T2 profiler bursts and updates the bottleneck label after each burst. The controller collects cheap evidence continuously and activates a short burst of heavy GPU profiling only when cheap evidence is insufficient to explain an anomaly. After each burst it evaluates whether the collected kernel-level evidence has stabilized into a diagnosis. When additional profiling would no longer change that diagnosis, the controller stops heavy tracing and returns to cheap-only monitoring.

1) Evidence Tiers: The controller organizes evidence into three tiers ordered by collection cost, summarized in Table I.

TABLE I
Evidence tiers used by the adaptive tracing controller.

Tier	Signals	Collection mode	Cost
T0	Request latency (p50, p95), throughput (req/s), queueing-delay proxy	Always-on, per request	Negligible
T1	GPU utilization, GPU memory utilization, SM clock, power draw	Always-on, NVML polling	Negligible
T2	Kernel execution timing, duration distribution, kernel launch frequency	Short profiler burst, Nsight Systems CLI	High

T0 and T1 signals are sampled continuously and aggregated into fixed-size time windows (10 s in all experiments). T2 evidence is collected only when the controller determines that T0/T1 evidence alone is insufficient to identify the bottleneck class. T2 collection is bounded to a short burst duration and does not run continuously.

2) Controller State Machine: The controller operates as a finite-state loop over evidence windows. Figure 1 illustrates the five states and the transitions between them.

In Evaluating, the controller classifies the bottleneck from the kernel summary (kernel count, total and mean kernel time, top kernel name) combined with the current T0/T1 window, then applies the active stopping policy to decide whether to stop or launch another burst.

3) Windowing and Aggregation: Evidence is aggregated over non-overlapping fixed-size time windows. Within each window the controller computes:

- **Latency statistics**: mean, p50, p95 request latency; ratio versus the initial-window baseline.
- **Throughput**: requests per second; ratio versus the initial-window baseline.

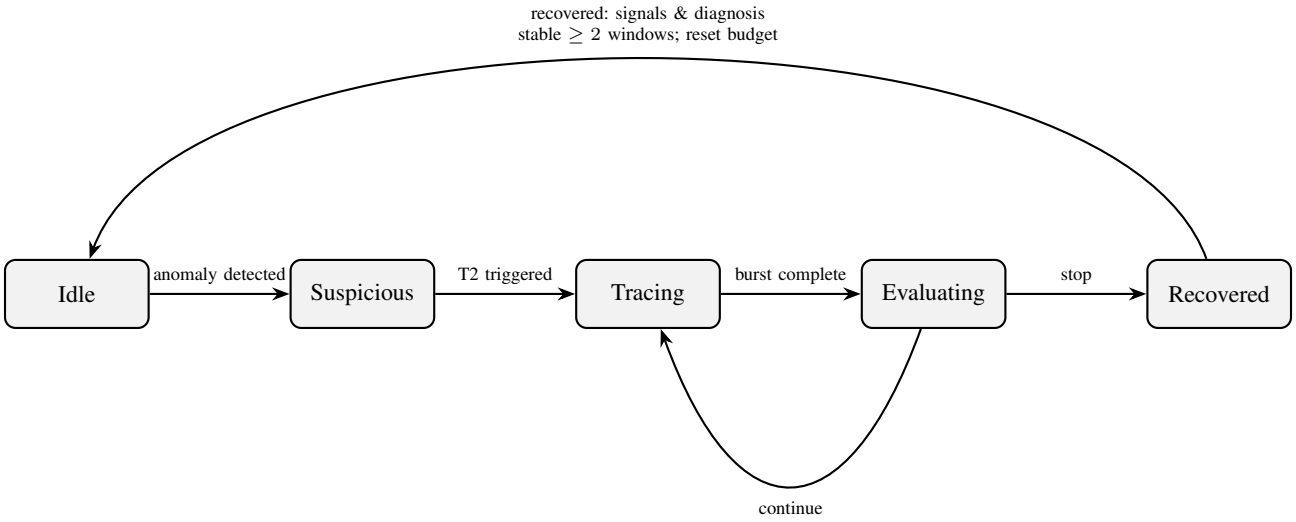


Fig. 1. Controller state machine. The controller escalates from IDLE through SUSPICIOUS and TRACING to EVALUATING, which either launches another burst (returning to TRACING) or de-escalates to RECOVERED; the controller returns to IDLE once cheap signals and diagnosis stability both confirm recovery.

- **Queueing delay proxy**: difference between request completion time and a minimum expected service time, aggregated at p95 across the window.
- **GPU utilization**: mean and standard deviation of NVML `utilization.gpu` samples taken every 0.5 s during the window.
- **GPU memory utilization**: mean NVML `utilization.memory` and used-memory fraction.
- **Diagnosis stability streak**: number of consecutive windows for which the top bottleneck class has remained unchanged.
- **Recovery streaks**: number of consecutive windows for which each recovery condition (latency, throughput, queueing delay, GPU utilization) has been satisfied.

The stability streak is computed only over concrete bottleneck labels. The ambiguous label `unknown_gpu_cause` does not increment the streak. A pre-burst cheap-label candidate may contribute to the streak only if it is later confirmed by the first T2 burst; otherwise the streak is reset. This rule allows a one-burst stop when cheap and kernel evidence agree, while preventing an ambiguous first burst from satisfying a $K = 2$ stability condition.

When T2 evidence is available for a window, the controller additionally records the kernel summary extracted from the Nsight Systems report: total kernel instances, total kernel time, mean kernel duration, kernel duration coefficient of variation (CV), and the name of the most time-consuming kernel.

4) Anomaly Detection: A window is marked suspicious when any of the following conditions holds:

1) **Latency regression**: the current-window p95 latency exceeds $1.5\times$ the initial-window p95 baseline.
2) **Sustained GPU saturation**: mean GPU utilization exceeds 85% for two or more consecutive windows while request latency is elevated.
3) **Unexplained service cost**: latency is elevated while throughput, queueing-delay proxy, GPU utilization, and memory use do not yield a stable cheap-label candidate.

Kernel launch rate is not used to trigger the first T2 burst because it is available only after profiler evidence has been collected. Once a burst exists, launch rate is used for bottleneck classification and for stop/continue decisions.

A suspicious window that already carries a stable bottleneck classification does not re-trigger T2 profiling. T2 is triggered only when the window is suspicious and the current diagnosis is absent, ambiguous, or unstable.

5) Tracing Budget: Each anomaly episode is capped at six profiler bursts. Budget exhaustion forces de-escalation and is logged as `budget_exhausted`. A budget-exhausted episode is treated as inconclusive for evidence-sufficiency claims rather than as a successful stop. In our experiments, no reported live run reached this limit.

B. Short-Burst Stopping Policies

A stopping policy controls when the controller de-escalates from heavy GPU tracing back to cheap-only monitoring. After each profiler burst, the controller invokes the active policy to decide whether to launch another burst or stop. We define six candidate policies that span the spectrum from aggressive cost reduction to conservative reliability, applying the same cost-awareness behind adaptive and volume-reducing tracing decisions for distributed request traces [15]–[19] to GPU kernel-level evidence instead.

1) Policy Definitions:

*a) Fixed Burst (*FB*).:* Collect exactly one profiler burst after the first suspicious window, then stop. FB is the most aggressive fixed-count policy: it commits to a diagnosis from a single burst and never continues, making it the cheapest option but brittle when that one burst is ambiguous.

*b) Repeated Fixed Burst (*RFB*).:* Collect a fixed number N of bursts (default $N = 3$), then stop regardless of diagnosis stability. RFB generalizes FB (which is the $N = 1$ case): it buys more evidence than a single burst but, like FB, ignores whether that evidence has converged. It serves as the evidence-blind, fixed-cost baseline against which the adaptive policies (SS, MU) are compared.

*c) Stability Stop (*SS*).:* Continue collecting bursts until the bottleneck class label has remained unchanged for K consecutive evidence windows (default $K = 2$), then stop. SS is evidence-aware: it collects more bursts when early windows are ambiguous and fewer when the label is clear from the start.

*d) Marginal Utility Stop (*MU*).:* MU stops after at least two concrete evidence windows when the newest bottleneck label matches the immediately preceding concrete label. It is more aggressive than SS because it requires one repeated label rather than a two-window stable streak under the full policy state. Because it requires only one match rather than K consecutive matches, it collects fewer bursts than SS but is more vulnerable to stopping on a transient agreement that later windows would contradict.

*e) Counter-Recovery Stop (*CR*).:* Continue collecting bursts until T0/T1 cheap signals return to their pre-anomaly baselines: specifically, until p95 request latency drops below $1.1\times$ the initial-window baseline and GPU utilization drops below 60%. If recovery does not occur before the budget is exhausted, the controller stops at budget. CR couples de-escalation to observable workload recovery rather than to diagnosis stability. It is useful when operators want tracing to continue until the suspicious episode has cleared, but by itself it does not guarantee that the bottleneck label has converged.

*f) Hybrid Stop (*HS*).:* Stop only when *both* conditions hold: (1) the diagnosis has been stable for K consecutive windows (as in SS), and (2) T0/T1 cheap signals show sustained recovery (as in CR). HS is the most conservative policy: it requires corroborating evidence from both the kernel-level diagnosis and the cheap runtime signals before committing to de-escalation. Neither condition alone is sufficient.

2) Policy Summary: Unless otherwise stated, the live automatic controller uses SS with $K = 2$ and the stability-streak rule defined above. RQ4 replays all six policies over the same saved evidence streams to isolate the stop/continue decision. Together, these policies span fixed-cost, diagnosis-stability, recovery-based, and hybrid stopping strategies; RQ4 compares their cost–reliability tradeoffs empirically.

3) Stopping Signals: Each policy is implemented in terms of one or more *stopping signals*: boolean or numeric per-window features that the controller evaluates after each burst. Table II lists the candidate signals studied in this paper. Section IV-F (RQ5) evaluates which signals are most reliable for predicting correct de-escalation points.

C. Baselines

The primary live baseline is manual fixed-window profiling (FW), because it matches the common practice of launching a profiler for a fixed duration after observing a slowdown. We also use offline equal-budget replay to isolate stop/continue decisions under equal short-burst budgets.

TABLE II
CANDIDATE STOPPING SIGNALS EVALUATED IN THIS PAPER.

Signal	Definition
`diagnosis_stable_2`	Top bottleneck class unchanged for ≥ 2 consecutive windows
`diagnosis_changed_from_prev`	Top bottleneck class differs from the immediately preceding window
`diagnosis_margin_clear`	Confidence margin between top and runner-up diagnosis exceeds threshold
`latency_recovered`	p95 latency below $1.1\times$ initial-window baseline
`throughput_recovered`	Throughput above $0.9\times$ initial-window baseline
`queue_delay_recovered`	p95 queueing delay proxy below recovery threshold
`gpu_util_recovered`	Mean GPU utilization below 60%
`memory_pressure_low`	GPU memory used fraction below 95%
`kernel_duration_stable`	Kernel duration CV change across successive bursts below 10%

*a) Manual Fixed-Window Profiling (*FW*).:* When an anomaly is detected, FW starts one Nsight Systems burst of fixed duration, collects the kernel summary, produces a diagnosis, and stops. The fixed-window duration is set longer than the automatic burst duration to model a conservative manual profiling window, approximately $1.6\times$ to $2.0\times$ the automatic burst length across experiments.

b) Offline Equal-Budget Replay.: To separate the stopping policy from the choice of a longer manual window, we replay fixed-budget baselines over the saved per-window evidence streams. The replay baselines include an equal-duration fixed burst, a repeated fixed burst with $N = 3$, and the adaptive stopping policies. Replay uses the same observed labels, suspicion flags, recovery signals, and window durations; it does not execute new Nsight profiling.

D. Implementation

1) Cheap Metric Collection (T0 and T1): T0 request metrics are collected by the load-generation client, which records per-request start/end timestamps, HTTP status, estimated prompt/output token counts, and scenario/seed labels. At the end of each evidence window the client aggregates per-request records into window-level statistics: p50 and p95 latency, throughput in requests per second, request success rate, queueing delay proxy at p95, and ratio fields versus the initial-window baseline.

T1 GPU metrics are collected by a background NVML polling thread running at 0.5 s intervals using `nvidia-ml-py`. At window close, the thread aggregates all samples taken during the window into mean and standard deviation for GPU utilization, GPU memory utilization, used memory in MiB, SM clock speed, and GPU power draw.

2) Heavy Metric Collection (T2): T2 evidence is collected using the Nsight Systems command-line interface (`nsys` 2024.6). We use Nsight Systems because it provides kernel timing and CUDA activity summaries with a command-line workflow; related work has also studied low-overhead

trace collection for GPU compute kernels [20]. In the live experiments, each T2 burst profiles a bounded workload segment with `nsys profile`. For a long-running deployment, the same design can be implemented with Nsight Systems capture ranges, where profiling starts and stops at explicit CUDA-profiler or NVTX range boundaries. This avoids the process-termination behaviour that caused CUDA kernel tables to be absent in reports when `nsys` was given an explicit duration limit on some GPU hosts. The resulting kernel summary feeds a bottleneck classification scheme in the spirit of top-down GPU performance analysis [12], [13].

After the burst completes, the controller runs `nsys stats report cuda_gpu_kern_sum` on the resulting `.nsys-rep` file and parses the CUDA GPU kernel summary into the window record: distinct kernel type count, total kernel instances, total and mean kernel execution time, duration CV, and the name of the most time-consuming kernel.

3) Bottleneck Classification and Expected Labels: The controller assigns each suspicious window a label from a deterministic, hand-specified vocabulary. The labels used in the accuracy evaluation are *controlled expected labels*: they come from workloads intentionally constructed for the experiment, not from independent human annotation of production incidents. Table III summarizes the visible classifier rules and the label used for agreement checks; `unknown_gpu_cause` counts as ambiguous.

The agreement metric is intentionally scoped. It measures whether the controller recovers the label expected from the designed workload, not whether it discovers an independently verified production root cause. To reduce circularity, the expected label is fixed from the workload configuration before profiler summaries are parsed. The classifier then uses only the tier-visible fields listed in the table. We therefore report expected-label agreement rather than root-cause accuracy.

4) Artifacts and Reproducibility: The artifact contains four parts: (1) workload configurations for the microbenchmarks and vLLM scenarios; (2) raw per-request and per-window CSVs; (3) processed Nsight kernel summaries used by the classifier; and (4) replay and analysis scripts that regenerate Tables V–VIII. Each run manifest records the seed, GPU type, driver version, CUDA version, vLLM version, Nsight Systems version, controller parameters, and active stopping policy.

IV. Evaluation

We evaluate the adaptive tracing controller across five research questions. RQ1 asks whether the controller can determine when sufficient diagnostic evidence has been collected. RQ2 asks how accurate automatic tracing is compared with manual fixed-window profiling. RQ3 measures the overhead introduced by the controller. RQ4 compares the six short-burst stopping policies. RQ5 identifies which runtime signals are most useful for deciding when to stop heavy GPU tracing.

A. Experimental Setup

Experiments run on two single-GPU cloud platforms: an NVIDIA L4 (24 GB, Ada Lovelace, `sm_89`) as the primary platform for all research questions, and an NVIDIA A100 (40 GB, Ampere, `sm_80`) for cross-GPU validation in RQ1. All vLLM scenarios use vLLM v0.10.2 [14], an open-source inference engine that implements PagedAttention for efficient KV-cache management and exposes an OpenAI-compatible HTTP API. The system serves `Qwen/Qwen2.5-7B-Instruct` [21], a 7-billion-parameter instruction-tuned model, in BFloat16 precision.

Unless otherwise stated, vLLM uses its default configuration and automatically allocates the KV cache from available GPU memory. This provides approximately 64,704 KV-cache tokens on the 24 GB L4 host and 334,016 tokens on the 40 GB A100 host. CUDA graph capture remains enabled throughout the experiments. Nsight Systems still exposes GPU operations with timestamps, so kernel execution time and duration distributions remain visible. However, CPU-side launch counts can change under graph replay, so launch-frequency results are interpreted as profiler-visible kernel activity rather than raw Python- or CUDA-API launch calls.

Controller and policy parameters are fixed across experiments as listed in Table IV. Each workload uses five repetitions for microbenchmarks or three seeded repetitions for vLLM experiments, with results reported as means.

To support reproducibility, the implementation, configuration files, and evaluation artifacts are available in our anonymous repository.[1]

1) Controller Parameters: Table IV lists the controller and policy parameters held constant across all experiments.

2) Repetitions and Seeds: Microbenchmark experiments use five independent repetitions per workload, while vLLM serving experiments use three seeded repetitions per scenario; all results report means across repetitions and, where relevant, 95% confidence interval half-widths.

B. RQ1: Can the Controller Determine When Sufficient Diagnostic Evidence Has Been Collected?

1) Metrics: We evaluate evidence sufficiency using trace-time saved, expected-label match rate, and burst-count stability, measured as the standard deviation (σ) of profiler bursts across repetitions. Trace-time saved is the reduction in total profiler-active duration relative to the fixed-window baseline. For the microbenchmarks, we predefine success as trace-time savings of at least 25%, expected-label match rate of at least 0.95, and burst-count standard deviation $\sigma \leq 0.25$. For vLLM serving, we report trace-time savings descriptively because the anomaly episode length and recovery behavior vary by scenario.

2) Microbenchmark Results: Table V reports the RQ1 microbenchmark results for both the L4 and A100 platforms across five repetitions per workload.

The controller saves between 31.6% and 48.5% of profiler duration across all six workload-platform combinations while preserving expected-label agreement under the controlled labeling setup. Burst count standard deviation is 0.0 in every case: per-repetition logs show exactly one profiler burst in all

[1] https://anonymous.4open.science/r/gpu_adaptive_tracing-C52C/

TABLE III
BOTTLENECK CLASSIFIER USED FOR CONTROLLED EXPECTED-LABEL AGREEMENT.

Class	Cheap signal pattern	Kernel evidence pattern	Assigned label
`compute_bound`	High GPU util.; latency elevated	GEMM/attention kernels dominate total time	`compute_bound`
`launch_overhead`	High request cost with many short operations	Launch rate $> 2\times$ baseline; short mean kernels	`launch_overhead_or_small_kernel`
`mixed`	High GPU util. and memory use	Both long kernels and high launch count	`mixed`
`healthy`	Baseline latency/throughput; no sustained queue	No anomalous kernel summary required	`vllm_healthy`
`queue_pressure`	High queue-delay proxy; normal prompt/output mix	Standard decode kernels; no long-context signature	`vllm_queue_pressure`
`long_prompt`	High prompt tokens; elevated first-token latency	Prefill/attention kernels dominate early window	`vllm_long_prompt`
`long_output`	High output tokens; sustained decode latency	Repeated decode/GEMM kernels across window	`vllm_long_output`
`compute_saturation`	High concurrency, high util., throughput drop	GEMM/attention kernels dominate total time	`vllm_compute_saturation`
`kv_cache_pressure`	Memory near KV-cache limit; long contexts	Paged-attention/cache activity with high memory use	`vllm_kv_cache_pressure`

TABLE IV
CONTROLLER AND POLICY PARAMETERS USED IN ALL EXPERIMENTS.

Parameter	Value	Notes
Evidence window size	10 s	All experiments
Automatic burst duration	≈10–12 s	Bounded by target process duration
Fixed-window burst duration	≈17–24 s	1.6×–2.0× automatic
Stability window count K	2	SS, HS
Repeated burst count N	3	RFB
Maximum tracing budget	6 bursts/episode	All policies
Latency recovery ratio	1.10	CR, HS
GPU utilization recovery threshold	60%	CR, HS
NVML polling interval	0.5 s	T1 collection

TABLE V
RQ1 MICROBENCHMARK RESULTS: AUTOMATIC VS. FIXED-WINDOW PROFILER DURATION (MEAN OVER 5 REPETITIONS). ALL ROWS PASS THE COMBINED SUCCESS CRITERION.

GPU	Workload	Auto (s)	Fixed (s)	Saved (%)	Match rate	Burst σ
A100	`compute_bound`	10.4	16.7	37.6	1.0	0.0
A100	`launch_overhead`	11.9	23.2	48.5	1.0	0.0
A100	`mixed`	10.2	16.8	39.2	1.0	0.0
L4	`compute_bound`	12.4	18.1	31.6	1.0	0.0
L4	`launch_overhead`	13.0	24.0	45.8	1.0	0.0
L4	`mixed`	12.0	18.5	35.4	1.0	0.0

30 repetitions. This reflects the microbenchmark design: each workload presents a single, unambiguous bottleneck signature from the first suspicious window. RQ2 and RQ4 (Section IV-E) show burst counts greater than one when the bottleneck class is less immediately separable. All rows pass the combined RQ1 success criterion.

Savings are consistently higher for `launch_overhead_or_small_kernel` than for `compute_bound` or `mixed` on both GPUs. This is expected: the kernel summary for a high-launch-rate workload reveals its bottleneck class (many short kernels) after a short burst, allowing the stability condition to be satisfied quickly, whereas compute-bound and mixed workloads require a slightly longer burst to produce a stable GEMM-dominated summary.

Per-condition savings are stable: 95% CI half-widths range from ± 0.5 to ± 2.7 points across the six combinations, an order of magnitude smaller than the 17-point spread between conditions.

3) vLLM Serving Results: On the `queue_pressure` vLLM serving scenario, across three seeded repetitions on the L4, the controller reduces mean profiler duration from 133.5 s (fixed-window) to 104.6 s (automatic), a 21.7% reduction (95% CI ± 4.0pp) while maintaining 100% request success in every repetition.

4) RQ1: Answer: The controller reduces profiler duration while preserving expected-label agreement in the controlled workloads. All six microbenchmark GPU/workload combinations meet the predefined RQ1 success criterion, with 31.6%–48.5% trace-time savings and zero burst-count variation. The vLLM `queue_pressure` case shows a smaller but still positive 21.7% reduction, so we treat it as supportive serving evidence rather than as a pass under the 25% microbenchmark threshold.

C. RQ2: How Accurate Is Automatic GPU Tracing Compared with Manual Profiling?

1) Metrics: We evaluate diagnosis accuracy using expected-label match rate, ambiguous or unknown rate, time-to-diagnosis, and automatic-versus-fixed-window disagreement rate. Expected labels are the controlled workload labels defined in Table III; they represent agreement with the experiment's designed bottleneck, not independent annotation of production incidents. Expected-label match rate is evaluated separately for automatic and fixed-window modes, with a success threshold of at least 0.95; the ambiguous or unknown rate has a target of at most 0.05. Time-to-diagnosis is the mean elapsed time from run start to the first window assigned the expected label. Disagreement measures the fraction of paired windows for which automatic and fixed-window modes assign different labels; disagreement alone is not an accuracy failure when both modes match the expected label in their respective suspicious windows.

2) Microbenchmark Results: Across all three L4 microbenchmark workloads and five repetitions each, automatic and fixed-window tracing achieved a 1.0 expected-label match rate and a 0.0 ambiguous rate. Both modes identified the expected bottleneck in the first evidence window, within approximately 5–6 s.

3) vLLM Multiclass Results: Table VI reports RQ2 accuracy for the vLLM multiclass long-run experiments (three

seeded repetitions per scenario, 402 per-window records total across six scenarios).

The RQ2 multiclass runs use an evaluation profiling mode that collects comparable profiler evidence for each scenario, including `healthy`, so that classifier agreement and profiler cost can be compared under the same table structure. This mode should not be read as the deployment behavior of the anomaly-triggered controller: in deployment, a healthy stream would remain in cheap-only monitoring unless it became suspicious.

Automatic and fixed-window tracing satisfy all expected-label criteria across the six vLLM serving scenarios (Table VI), with a 1.0 expected-label match rate, zero ambiguous rate, and diagnosis in the first 10 s evidence window.

Non-zero disagreement rates reflect sequence-level differences rather than diagnosis failures. `long_prompt` is the only vLLM scenario with non-zero disagreement (0.25), because automatic mode stops collecting bursts earlier, so automatic and fixed-window tracing do not always assign profiler-enriched labels to the same windows; both modes still assign the correct expected label on every suspicious window.

Across all six vLLM scenarios, automatic mode preserves the fixed-window expected-label agreement while profiling for 23.8% less time and recording 47.1% fewer kernel instances.

4) RQ2: Answer: Automatic GPU tracing preserves expected-label agreement under the controlled labeling setup: the match rate is 1.0 and the ambiguous rate is 0.0 across the microbenchmark workloads and the six vLLM evaluation scenarios. In live L4 vLLM runs, automatic mode reaches this agreement using 23.8% less measured profiler time and recording 47.1% fewer kernel instances than the conservative manual fixed-window baseline.

D. RQ3: What Runtime Perturbation and Profiler-Cost Savings Does the Controller Introduce?

We study overhead from two complementary angles. The first is *profiler cost*: how much profiler-active time and how many kernel instances are recorded by adaptive tracing compared with fixed-window profiling? The second is *runtime perturbation*: does running the cheap controller metrics alongside vLLM introduce measurable degradation in request latency or throughput?

1) Metrics:

- **Trace-time saved (%) and kernel instances avoided (%).** Reduction in profiler-active duration and in the number of kernel instances recorded by the profiler, relative to the fixed-window baseline.
- **p95 latency regression (%).** Change in p95 request latency between cheap-metrics mode and no-profiler mode, computed as $(p95_{\text{cheap}} - p95_{\text{none}})/p95_{\text{none}} \times 100$. Positive values indicate regression; negative values indicate improvement.
- **Throughput change (%).** Change in requests per second between cheap-metrics mode and no-profiler mode.
- **Request success rate.** Fraction of requests that completed successfully. Must remain at 100%.

The RQ3 success criteria are: positive trace-time savings, positive kernel-instance reduction, p95 latency regression $\leq 5\%$, and request success rate of 100%.

2) Profiler-Cost Overhead: Combining the RQ1 and RQ2 artifacts (Tables V–VI), automatic tracing reduces profiler duration by 18.9%–48.5% and captured kernel instances by 42.9%–78.2% across all workloads, relative to the fixed-window baseline and while preserving expected-label agreement. Microbenchmark workloads show higher savings than vLLM scenarios because their bottleneck classes stabilize more quickly.

3) Dedicated Runtime Perturbation: To isolate the cost of the cheap controller metrics, we compare a *no-profiler* baseline against a *cheap-metrics-only* mode (NVML polling and windowed metric aggregation, no Nsight profiling), randomizing which mode runs first in each repetition to control for run-order effects, across five randomized repetitions of `queue_pressure` and three of `healthy`. All rows pass the RQ3 criteria.

The cheap controller metrics introduce no measurable p95 latency regression on either scenario. For `queue_pressure` the mean p95 latency is 3.6% *lower* in cheap-metrics mode (5 reps, 95% CI ± 6.3 points), and for `healthy` the regression is $+0.6\%$ (3 reps, 95% CI ± 1.9 points), well within the 5% threshold. Both intervals are wide given the small repetition counts, but even `queue_pressure`'s upper bound ($+2.7\%$) clears the threshold with margin. Request success remains at 100% in both scenarios and both modes.

Throughput change is small (-0.5% to -1.0%) but more sensitive to run order than p95 latency; we report it as context rather than as a hard criterion.

4) RQ3: Answer: Automatic tracing reduces profiler cost substantially relative to fixed-window profiling without degrading serving performance: profiler duration drops by 18.9%–48.5% and captured kernel instances by 42.9%–78.2%. The cheap controller metrics stay below the 5% p95-latency regression threshold in both perturbation scenarios: `queue_pressure` changes by -3.6% on average and `healthy` by $+0.6\%$. Because the confidence intervals are wide, we interpret this as no measurable regression in these runs rather than as proof of zero overhead.

E. RQ4: Which Short-Burst GPU Tracing Policies Work Best?

1) Evaluation Approach: RQ4 compares the six stopping policies via offline replay over real L4 vLLM window evidence, avoiding additional expensive Nsight executions per policy. Each policy is replayed over the same per-window evidence streams, so differences in outcome are attributable solely to the stopping condition rather than to workload variation or profiler noise. This assumes a policy's stopping choice does not meaningfully change later windows' evidence; we expect this confound to be small since T0/T1 collection is always-on and policy-independent, and RQ3 shows that cheap metrics remain within the 5% p95-latency regression threshold.

TABLE VI
RQ2 VLLM MULTICLASS ACCURACY (3 SEEDS PER SCENARIO, L4). ALL SIX SCENARIOS MEET THE EXPECTED-LABEL CRITERIA IN BOTH MODES. AUTO TRACE AND FIXED-WINDOW TRACE COLUMNS REPORT TOTAL PROFILER DURATION ACROSS ALL REPETITIONS.

Scenario	Mode	Match	Ambig.	TTD (s)	Disagree	Trace (s)	Kernels
`healthy`	Auto	1.0	0.0	10.0	0.00	301.3	730K
	Fixed	1.0	0.0	10.0		391.9	1380K
`queue_pressure`	Auto	1.0	0.0	10.0	0.00	309.7	748K
	Fixed	1.0	0.0	10.0		403.4	1380K
`long_prompt`	Auto	1.0	0.0	10.0	0.25	290.4	732K
	Fixed	1.0	0.0	10.0		381.0	1380K
`long_output`	Auto	1.0	0.0	10.0	0.00	291.0	729K
	Fixed	1.0	0.0	10.0		397.1	1380K
`compute_saturation`	Auto	1.0	0.0	10.0	0.00	336.8	757K
	Fixed	1.0	0.0	10.0		415.1	1380K
`kv_cache_pressure`	Auto	1.0	0.0	10.0	0.00	329.4	731K
	Fixed	1.0	0.0	10.0		449.3	1380K

We use two replay datasets. The *stable replay* uses the real automatic-mode windows from the six vLLM serving scenarios across three seeds (18 streams). In this dataset the correct bottleneck label appears in the first suspicious window for every stream, making it an easy diagnostic setting. The *ambiguity stress replay* uses the same 18 streams with the first suspicious window in each stream relabeled as unknown, modeling the realistic case where a single burst is insufficient. The stress dataset is the primary discriminating evaluation because it separates policies that can handle ambiguous early windows from those that cannot.

2) Equal-Budget Baseline Replay: The fixed-window results above compare against a conservative manual window that is longer than the automatic burst. To check that the conclusion is not only an artifact of this longer baseline, we replayed equal-budget stopping baselines over the saved L4 and A100 per-window streams. Equal-duration fixed bursts and repeated fixed bursts recovered the expected label in these stable streams, but they stopped while suspicious windows remained: re-escalation rate was 1.00 for equal-duration fixed burst on both GPUs, 0.83 for repeated fixed burst on L4, and 1.00 for repeated fixed burst on A100. Counter-Recovery Stop and Hybrid Stop eliminated re-escalation on both GPUs, selecting about four short trace windows per stream. Replay does not create new Nsight reports or kernel counts; it isolates stop/continue decisions under equal trace-window budgets.

3) Metrics: We report mean top-1 accuracy, premature-stop rate, re-escalation-needed rate (the fraction of streams with suspicious windows remaining after the policy stops), mean heavy-trace duration, and duration saved relative to the Repeated Fixed Burst baseline ($N = 3$). These are combined into a composite score weighted 40% accuracy, 25% premature-stop avoidance, 20% re-escalation avoidance, and 15% cost-efficiency.[2]

[2]Composite score $= 0.40 \times$ accuracy $+ 0.25 \times (1 -$ premature-stop rate$) + 0.20 \times (1 -$ re-escalation rate$) + 0.15 \times$ cost-efficiency, where cost-efficiency is the normalised inverse of mean trace duration.

TABLE VII
RQ4 AMBIGUITY STRESS POLICY RANKING (18 STREAMS, FIRST WINDOW RELABELED AS UNKNOWN). FB FAILS ENTIRELY; MULTI-WINDOW POLICIES RECOVER TO TOP-1 ACCURACY OF 1.0.

Rank	Policy	Top-1	Premature	Re-esc.	Trace (s)	Saved (%)	Score
1	CR	1.0	0.0	0.0	34.9	−16.3	0.868
2	HS	1.0	0.0	0.0	34.9	−16.3	0.868
3	MU	1.0	0.0	0.8	30.0	0.0	0.720
4	RFB	1.0	0.0	0.8	30.0	0.0	0.720
5	SS	1.0	0.0	0.8	30.0	0.0	0.720
6	FB	0.0	1.0	1.0	10.0	+66.7	0.112

4) Stable Replay Results: The stable replay is not the discriminating condition: every policy reaches top-1 accuracy of 1.0 with zero premature stops. CR and HS avoided re-escalation entirely but required 34.9 s of tracing per episode, whereas FB used 10.0 s but left suspicious windows after stopping. MU and SS were Pareto-dominated by FB: they incurred twice the tracing cost (20.0 s) without improving accuracy or re-escalation rate. RFB fell between these extremes (30.0 s, re-escalation rate 0.8) and remained Pareto-efficient only because no other policy matched its particular cost–reliability point.

5) Ambiguity Stress Replay Results: Table VII reports policy rankings on the ambiguity stress dataset, which is the primary discriminating evaluation.

The stress dataset clearly separates one-burst and multi-burst policies. FB fails completely: its top-1 accuracy drops to 0.0 and its premature-stop rate rises to 1.0, because it commits to the ambiguous first-window unknown label and never collects the additional burst needed to reach the concrete diagnosis. We do not use Pareto status as a selection criterion in the stress replay because cost-only efficiency can make a failing policy appear attractive. Instead, policies must first satisfy the correctness filter: top-1 accuracy of 1.0 and premature-stop rate of 0.0.

All five multi-window policies recover to a top-1 accuracy of 1.0 and a premature-stop rate of 0.0. CR and HS again rank highest with a re-escalation rate of 0.0. CR waits for workload recovery, while HS waits for both workload re-

covery and diagnosis stability. Both therefore avoid stopping while suspicious windows remain, but HS gives the stronger evidence-sufficiency guarantee.

MU, SS, and RFB are tied at rank 3–5 with a re-escalation rate of 0.83 and a mean trace duration of 30 s. MU and SS are preferable to RFB since they stop on evidence convergence rather than a fixed burst count.

6) RQ4: Answer: No policy dominates on all dimensions. For safety-first deployments, HS is the most defensible default because it requires both a stable diagnosis and workload recovery. CR achieves the same zero re-escalation rate in our replay, but its recovery-only condition gives a weaker diagnosis-sufficiency guarantee. For budget-sensitive deployments, SS and MU avoid the one-burst failure of FB while tracing less than the recovery-gated policies, but they accept a re-escalation rate of 0.83 in the ambiguity stress replay. FB should not be used as a standalone policy when the first burst can be ambiguous.

F. RQ5: Which Runtime Signals Are Most Useful for Deciding When to Stop Heavy GPU Tracing?

1) Evaluation Approach: RQ5 evaluates the nine candidate stopping signals defined in Section III-B on two datasets.

Real enriched L4 runs. 18 vLLM serving runs (6 scenarios $\times$ 3 seeds, 60 s per run) collected with the enriched window schema that includes non-sparse baseline ratio fields, recovery streak counters, and diagnosis confidence and margin fields. These runs provide a realistic serving signal distribution for evaluating which signals identify correct stopping points.

Ambiguity stress replay. The same stress dataset used in RQ4 (first suspicious window relabeled as unknown). This dataset tests whether a signal fires on ambiguous stopping points as well as correct ones; a signal that cannot distinguish between the two is not safe to use as a standalone stopping criterion.

We require a standalone stopping signal to be defined in both evaluation datasets. This availability requirement is separate from precision and F1: a signal that depends on fields absent from one evidence schema, or on trace history that is unavailable at the stopping point, cannot serve as a default standalone stop rule. A standalone stopping signal must be defined at the point where the controller makes a stop decision. A signal is considered valid as a standalone stopping criterion if it is available in both datasets and satisfies all three of the following criteria simultaneously: (1) precision ≥ 0.90 on both datasets, (2) correct-stop F1 ≥ 0.70 on the stress replay, and (3) ambiguous-stop F1 ≤ 0.10 on the stress replay. Criterion (3) is the strictest filter: a signal that fires reliably on correct windows but also fires on ambiguous windows would cause the controller to stop too early when the diagnosis has not yet converged.

2) Signal Ranking Results: Table VIII reports precision, recall, and F1 for each candidate signal across the two evaluation datasets, together with the standalone-signal verdict.

`diagnosis_stable_2` is the only signal satisfying all three standalone-signal criteria (Table VIII).

3) Analysis of Individual Signal Groups:

a) Diagnosis-evidence signals.: `diagnosis_stable_2` is the strongest and most defensible signal because it is derived directly from the kernel-level evidence the controller has already collected. It requires no external baseline calibration and does not depend on the workload's utilization or request rate returning to a specific threshold. Its main limitation is recall: on the real enriched runs it fires on 86% of correct stopping points rather than all of them, meaning it occasionally requires one additional window before triggering. `diagnosis_margin_clear` is promising on the real enriched runs (F1 0.853), but it requires an enriched diagnosis schema with margin fields. The ambiguity stress replay uses the minimal diagnosis schema, so this signal does not satisfy the cross-dataset availability requirement for a default standalone stop rule.

`diagnosis_changed_from_prev` has 1.000 precision but low recall (F1 0.635 on real enriched runs). It is better understood as a *continuation signal* than a stopping signal: when the diagnosis changes from one window to the next, the controller should continue tracing rather than stop. This is the inverse of its intended stopping role.

b) Recovery signals.: `latency_recovered` and `queue_delay_recovered` achieve high F1 on real enriched runs (1.000 and 0.794) but fail the ambiguous-stop criterion with stress replay F1 values of 0.414. The stress transformation only relabels the diagnosis field, so latency can appear recovered while the diagnosis remains unknown. These signals are useful within CR and HS, but unsafe as standalone stopping criteria.

`throughput_recovered` shows a similar pattern: high F1 on real enriched runs (0.938) but fails the ambiguity criterion (stress F1 0.333).

c) GPU counter signals.: At default thresholds (60% util, 95% memory), both score 0.000 since utilization never drops that low (observed range: 92.4–99.4% util, 96.7–97.7% memory). Recalibrating util to 99% raises `gpu_util_recovered`'s recall to 0.953, but it then fires on 100% of ambiguous windows (F1 0.419), violating the safety criterion; `memory_pressure_low` stays near-uninformative (recall 0.054) even at 97.5%.

d) Kernel duration stability.: `kernel_duration_stable` is a second-burst consistency check rather than a first stop signal. By definition, it requires two consecutive T2 summaries and is unavailable at one-burst stopping points. It is therefore unsuitable as a default standalone criterion, although it may be useful as a supporting condition in policies that always collect at least two bursts.

4) RQ5: Answer: Among the signals available across both datasets, `diagnosis_stable_2` is the safest standalone stopping criterion: it reaches 1.000 precision and 0.925 F1 on real enriched L4 runs and does not fire on ambiguous stopping points in the stress replay. Recovery signals are useful as supporting conditions in HS and CR, but they are

TABLE VIII

RQ5 STOPPING SIGNAL RANKING. "UNDEF." MEANS THE SIGNAL IS NOT DEFINED AT THE RELEVANT STOPPING POINT BECAUSE ITS REQUIRED EVIDENCE IS UNAVAILABLE, NOT THAT AN EXPERIMENT WAS OMITTED. A STANDALONE SIGNAL MUST BE DEFINED WHEN THE CONTROLLER MAKES A STOP DECISION AND MUST SATISFY THE PRECISION, CORRECT-STOP, AND AMBIGUITY-STOP CRITERIA.

	Real enriched			Stress replay			
Signal	**Prec.**	**Rec.**	**F1**	**Prec.**	**F1 (correct)**	**F1 (ambig.)**	**Standalone?**
`diagnosis_stable_2`	1.000	0.860	0.925	1.000	0.786	0.000	**yes**
`latency_recovered`	1.000	1.000	1.000	0.739	0.850	0.414	no
`throughput_recovered`	1.000	0.884	0.938	0.778	0.800	0.333	no
`queue_delay_recovered`	1.000	0.659	0.794	0.739	0.850	0.414	no
`diagnosis_margin_clear`	1.000	0.744	0.853	undef.	undef.	undef.	no
`diagnosis_changed_from_prev`	1.000	0.465	0.635	1.000	0.522	0.000	no
`gpu_util_recovered`	0.000	0.000	0.000	0.000	0.000	0.000	no
`memory_pressure_low`	0.000	0.000	0.000	0.000	0.000	0.000	no
`kernel_duration_stable`	undef.	undef.	undef.	undef.	undef.	undef.	no

unsafe alone because they can fire while the diagnosis is still unknown. Signals that require enriched diagnosis fields or multiple consecutive T2 summaries do not satisfy the availability requirement for a default standalone rule. GPU counter signals are uninformative under the saturated serving conditions studied here.

V. DISCUSSION AND SCOPE

This paper does not claim a learned stopping policy, a universal GPU bottleneck classifier, or validation on independently labeled production incidents. The controller is a deterministic debugging aid for deciding when to collect profiler evidence, the labels are controlled expected labels from designed workloads, and replay baselines isolate stop/continue decisions without producing new Nsight kernel counts. The intended use is therefore practical triage: reducing how often a developer or operator must choose profiler windows manually, while still surfacing the kernel-level evidence needed for diagnosis. These choices make the evaluation reproducible and targeted, but they bound the claims to the studied model, serving stack, GPUs, profiler, and workloads.

The policy choice depends on the operational goal. If the priority is avoiding repeated profiler activation during an incident, HS is the safest default because it requires both stable diagnosis and recovery. If the priority is minimizing trace volume after a clear diagnosis, SS is the better default. FB should be used only as a fixed-cost baseline, not as a deployment policy, because it fails when the first evidence window is ambiguous.

VI. THREATS TO VALIDITY

Several factors limit generalizability. First, the RQ2 equal-budget comparison and the RQ4/RQ5 policy analyses use offline replay over saved per-window evidence. Replay isolates the stopping decision, but it assumes earlier stopping would not materially change later T0/T1 evidence and cannot produce new Nsight kernel counts. Second, the live implementation profiles bounded workload segments; a production deployment would need robust capture-range integration for long-running serving processes. Third, the ambiguity stress replay is synthetic: it tests whether policies handle an inconclusive first window, but it is not a substitute for naturally occurring ambiguous incidents. Fourth, expected-label match measures agreement with the controlled workload labels in Table III; it should not be read as independently validated production root-cause accuracy. Externally, the evaluation uses one model family, one serving framework, one profiler, and two GPU platforms, so results may differ for other architectures, serving stacks, profilers, loads, or GPU generations. Repetition counts are modest, and the RQ4 composite-score weights are a design choice.

VII. CONCLUSION AND FUTURE WORK

This paper presented an adaptive GPU tracing controller for LLM-serving debugging support. The controller keeps cheap request and GPU metrics on continuously, escalates to bounded Nsight Systems bursts when cheap evidence is insufficient, and stops heavy collection when the bottleneck label has stabilized. In controlled workloads on L4 and A100 GPUs, adaptive tracing reduced profiler duration by 18.9%–48.5% and captured kernel instances by 42.9%–78.2% relative to fixed-window profiling while preserving expected-label agreement. The policy replay shows why stopping logic matters: one fixed burst fails under ambiguous first-window evidence, while recovery-gated policies avoid re-escalation at higher tracing cost. The signal analysis identifies two-window diagnosis stability as the safest evaluated standalone stopping signal. These results support treating GPU profiling as a runtime observability-control problem, with claims scoped to the studied model, serving stack, GPUs, profiler, and controlled labels.

Future work should evaluate the controller on independently labeled production incidents and extend it to budget-aware policies for multi-GPU, multi-model, and multi-tenant serving with capture-range integration.

REFERENCES

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[2] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills," 2023. [Online]. Available: https://arxiv.org/abs/2308.16369

[3] J. Mace, R. Roelke, and R. Fonseca, "Pivot tracing: Dynamic causal monitoring for distributed systems," *ACM Transactions on Computer Systems (TOCS)*, vol. 35, no. 4, pp. 1–28, 2018.

[4] L. Zhang, Z. Xie, V. Anand, Y. Vigfusson, and J. Mace, "The benefit of hindsight: Tracing {Edge-Cases} in distributed systems," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 321–339.

[5] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure (2010)," 2010.

[6] R. Fonseca, G. Porter, R. H. Katz, S. Shenker, and I. Stoica, "X-trace: a pervasive network tracing framework," in *Proceedings of the 4th USENIX Conference on Networked Systems Design & Implementation*, ser. NSDI'07. USA: USENIX Association, 2007, p. 20.

[7] J. Kaldor, J. Mace, M. Bejda, E. Gao, W. Kuropatwa, J. O'Neill, K. W. Ong, B. Schaller, P. Shan, B. Viscomi, V. Venkataraman, K. Veeraraghavan, and Y. J. Song, "Canopy: An end-to-end performance tracing and analysis system," in *Proceedings of the 26th Symposium on Operating Systems Principles*, ser. SOSP '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 34–50. [Online]. Available: https://doi.org/10.1145/3132747.3132749

[8] H. Huang, X. Zhang, P. Chen, Z. He, Z. Chen, G. Yu, H. Chen, and C. Sun, "Trastrainer: Adaptive sampling for distributed traces with system runtime state," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 473–493, 2024.

[9] M. Toslali, S. Qasim, S. Parthasarathy, F. A. Oliveira, H. Huang, G. Stringhini, Z. Liu, and A. K. Coskun, "An online probabilistic distributed tracing system," 2024. [Online]. Available: https://arxiv.org/abs/2405.15645

[10] O. Villa, M. Stephenson, D. Nellans, and S. W. Keckler, "NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019. [Online]. Available: https://research.nvidia.com/publication/2019-10_nvbit-dynamic-binary-instrumentation-framework-nvidia-gpus

[11] K. Zhou, L. Adhianto, J. Anderson, A. Cherian, D. Grubisic, M. Krentel, Y. Liu, X. Meng, and J. Mellor-Crummey, "Measurement and analysis of gpu-accelerated applications with hpctoolkit," *Parallel Computing*, vol. 108, p. 102837, 2021.

[12] K. Zhou, M. W. Krentel, and J. Mellor-Crummey, "Tools for top-down performance analysis of gpu-accelerated applications," in *Proceedings of the 34th ACM International Conference on Supercomputing*, 2020, pp. 1–12.

[13] Y. Hao, N. Jain, R. Van der Wijngaart, N. Saxena, Y. Fan, and X. Liu, "Drgpu: A top-down profiler for gpu applications," in *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*, 2023, pp. 43–53.

[14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[15] P. Las-Casas, G. Papakerashvili, V. Anand, and J. Mace, "Sifter: Scalable sampling for distributed traces, without feature engineering," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC '19)*. ACM, 2019, pp. 312–324. [Online]. Available: https://doi.org/10.1145/3357223.3362736

[16] Z. Chen, Z. Jiang, Y. Su, M. R. Lyu, and Z. Zheng, "Tracemesh: Scalable and streaming sampling for distributed traces," 2024. [Online]. Available: https://arxiv.org/abs/2406.06975

[17] H. Huang, C. Chen, K. Chen, P. Chen, G. Yu, Z. He, Y. Wang, H. Zhang, and Q. Zhou, "Mint: Cost-efficient tracing with all requests collection via commonality and variability analysis," 2024. [Online]. Available: https://arxiv.org/abs/2411.04605

[18] Z. Chen, J. Pu, and Z. Zheng, "Tracezip: Efficient distributed tracing via trace compression," 2025. [Online]. Available: https://arxiv.org/abs/2502.06318

[19] J. Zhang, R. Ferydouni, A. Montana, D. Bittman, and P. Alvaro, "3milebeach: A tracer with teeth," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3472883.3486986

[20] S. Darche and M. R. Dagenais, "Low-overhead trace collection and profiling on GPU compute kernels," *ACM Transactions on Architecture and Code Optimization*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3649510

[21] Qwen Team, "Qwen2.5 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2412.15115